Week 7 Assignment: Store and Retrieve Data

An n8n AI Agent with Postgres-backed memory and OpenRouter as the LLM API

1. Introduction

This assignment asked for a workflow that stores data to a database, defines a schema, and retrieves data through an API. Rather than building three disconnected demo steps, I built one working system where all three requirements happen naturally as part of real usage: a chat-based AI Agent ("Tolu Agent") that remembers the conversation (store), is backed by a defined table structure (schema), and both calls an external LLM API and exposes tool APIs to fetch live data (retrieve).

The workflow is triggered by "When chat message received" and centers on an AI Agent node with three components attached: an OpenRouter Chat Model (the reasoning engine), Postgres Chat Memory (the database), and six RSS Read tool nodes (external data sources the agent can call on demand).

2. Schema

The database component is the Postgres Chat Memory node, configured with Table Name n8n_chat_histories and a Context Window Length of 5 (the model receives the last 5 stored interactions as context on each turn). Rather than me hand-writing insert/select logic, this node manages the table automatically, using the standard LangChain Postgres chat-history structure:

```
CREATE TABLE n8n_chat_histories (
  id         SERIAL PRIMARY KEY,
  session_id VARCHAR NOT NULL,
  message    JSONB NOT NULL
);
```

Column	Type	Purpose
id	SERIAL	Auto-incrementing row identifier.
session_id	VARCHAR	Groups messages into one conversation — the Postgres node's Session ID is pulled straight from the connected Chat Trigger node, e.g. c5806838ae79459ead121676588a0639.
message	JSONB	Stores the full LangChain message object (constructor type — HumanMessage or AIMessage — plus content), so the agent can reconstruct conversation order and context on the next turn.

This is a deliberate design choice worth stating plainly: I used n8n's managed schema instead of a hand-rolled table because the assignment's real goal — demonstrating that I understand how data is structured and persisted — is better shown by explaining this schema accurately than by reinventing an equivalent table by hand.

3. Workflow Walkthrough in n8n

Overview

Before drilling into each node's configuration, here's the workflow as a whole and a live execution of it, so the individual settings below can be read in context.

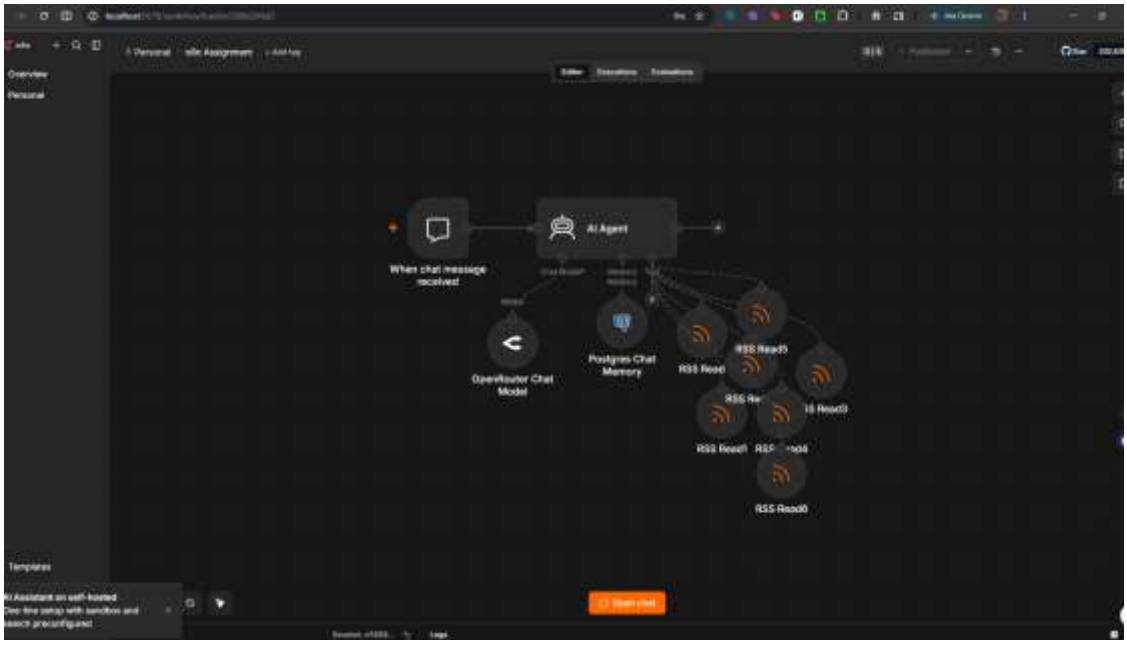


Figure 1. The completed AI Agent workflow — Chat Trigger, OpenRouter model, Postgres memory, and six RSS Read tools.



Figure 2. A live execution — the chat panel and execution log confirm the agent, model, and memory nodes all completed successfully (6.815s, 657 tokens).

Step 1 — Trigger: Chat Trigger

"When chat message received" is configured as a Hosted Chat, made publicly available with no authentication, at a webhook URL unique to this workflow. Its Initial Message introduces the agent by name ("Hi there! My name is Tolu Agent."), and custom CSS variables theme the chat widget's colors. Each

incoming message produces an item with an action (sendMessage), a sessionId, and the chatInput text — this sessionId is what threads the whole conversation together downstream.

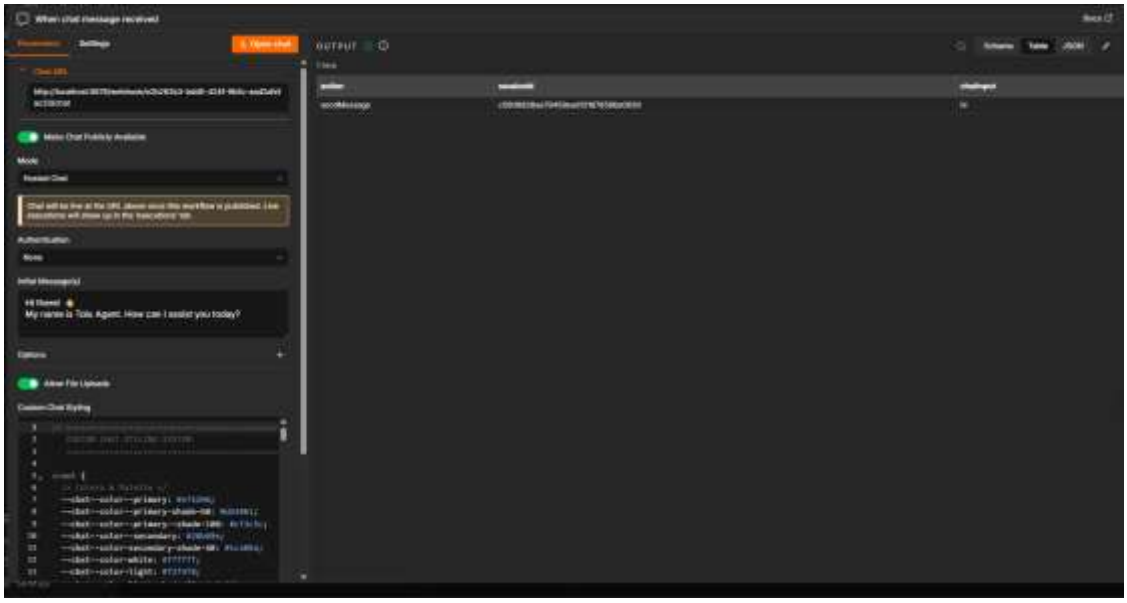


Figure 3. Chat Trigger configuration — hosted, public chat with a custom initial message and styling.

Step 2 — AI Agent (orchestrator)

The AI Agent node takes its Prompt (User Message) from the connected Chat Trigger's chatInput field via the expression {{ $json.chatInput }}. It exposes three typed connection points — Chat Model, Memory, and Tool — each wired to a different node below, so the agent can reason, remember, and act in the same turn. In the run shown, the output greets the user and offers to read an RSS feed, and the panel notes that no tools were used on that particular turn — confirming tool calls are conditional, not forced.

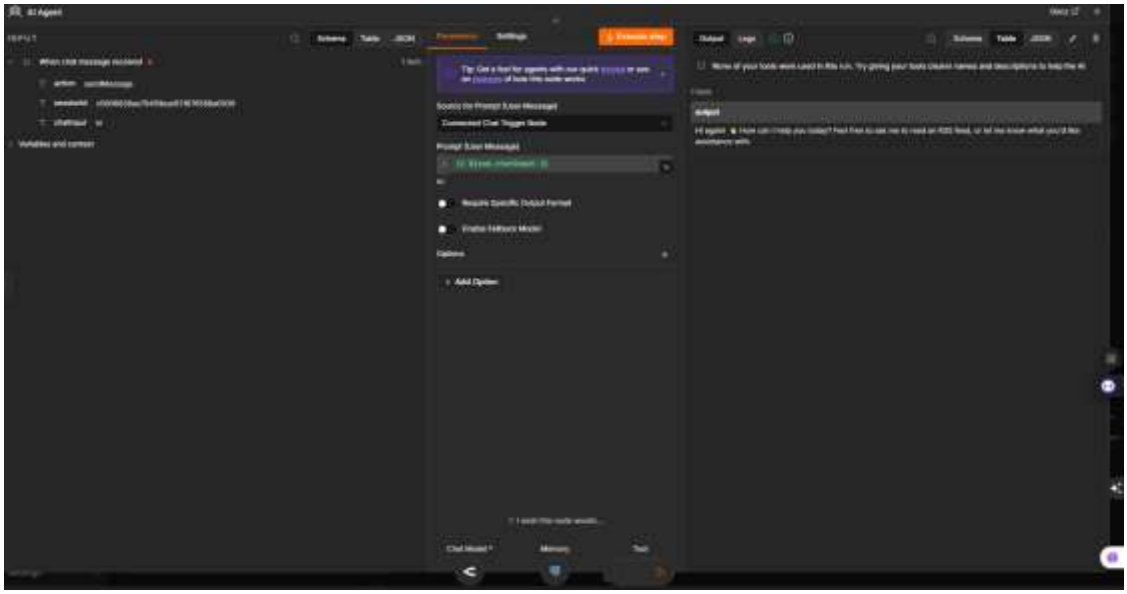


Figure 4. AI Agent configuration — prompt sourced from the Chat Trigger, with Chat Model / Memory / Tool slots below.

Step 3 — Chat Model: OpenRouter

The OpenRouter Chat Model node supplies the LLM, using the openrouter/free model and an OpenRouter credential whose base URL is set to https://openrouter.ai/api/v1 (OpenRouter mirrors the OpenAI API shape,

so the node authenticates with an API key the same way an OpenAI credential would). Every message the agent needs to reason over — including prior turns pulled in from memory — is sent to this endpoint, and the reply, along with token usage, is returned to the agent.

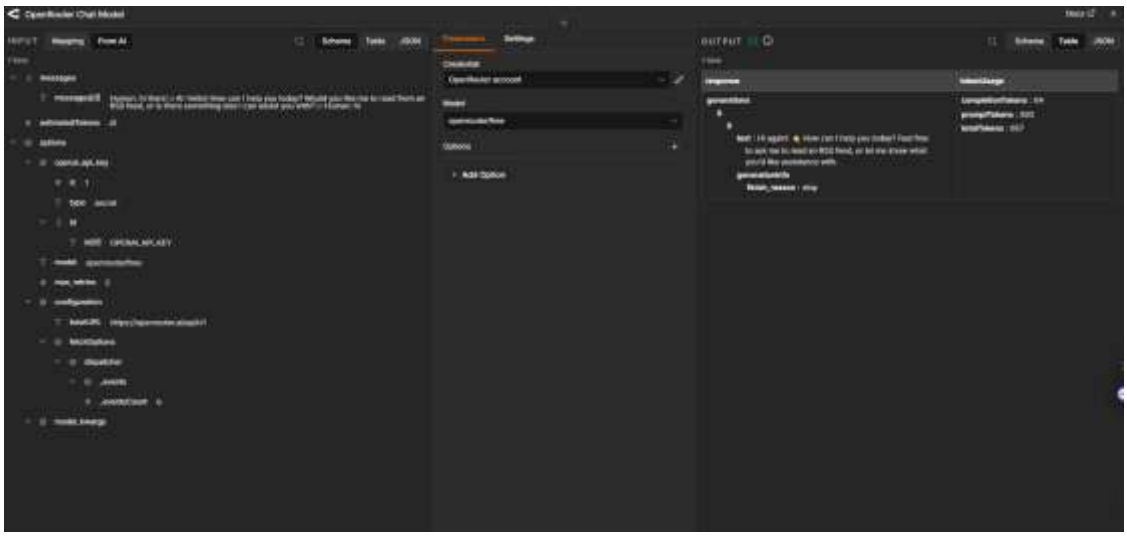


Figure 5. OpenRouter Chat Model — model, credential, and the returned response with token usage.

Step 4 — Memory: Postgres Chat Memory

Wired into the Memory slot, this node writes each human/AI message pair into the n8n_chat_histories table (Section 2) and reads past messages back in on the next turn, keyed by the session ID from the Chat Trigger. The output panel shows this concretely: on run 2 of 2, it lists three stored messages in order — the constructor type and content of each — confirming both the write (this turn's message) and the read (prior turns coming back as context) are happening. This is the "send data to database" requirement fulfilled as a natural side-effect of chatting, not a separate demo step.

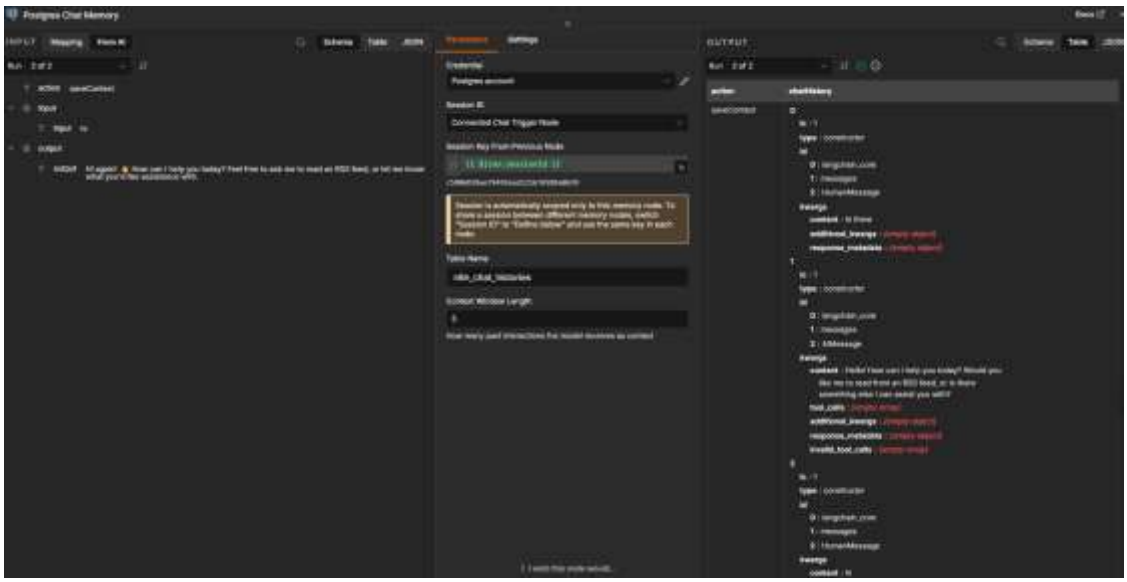


Figure 6. Postgres Chat Memory — table name, session ID source, and the stored chat history returned on read.

Step 5 — Tools: six RSS Read nodes

Six RSS Read nodes (RSS Read, RSS Read1–RSS Read6) are attached as Tools, each pointed at a different feed URL — for example RSS Read5 reads a YouTube channel's video feed

(youtube.com/feeds/videos.xml?...). The agent decides at runtime whether a user's request needs one of these — e.g. “read me the latest videos” — and if so, calls the matching node, which performs an HTTP GET against that feed's URL and returns parsed items back into the conversation. This is the “retrieve data via API” requirement, satisfied through tool-calling rather than a manually triggered request.

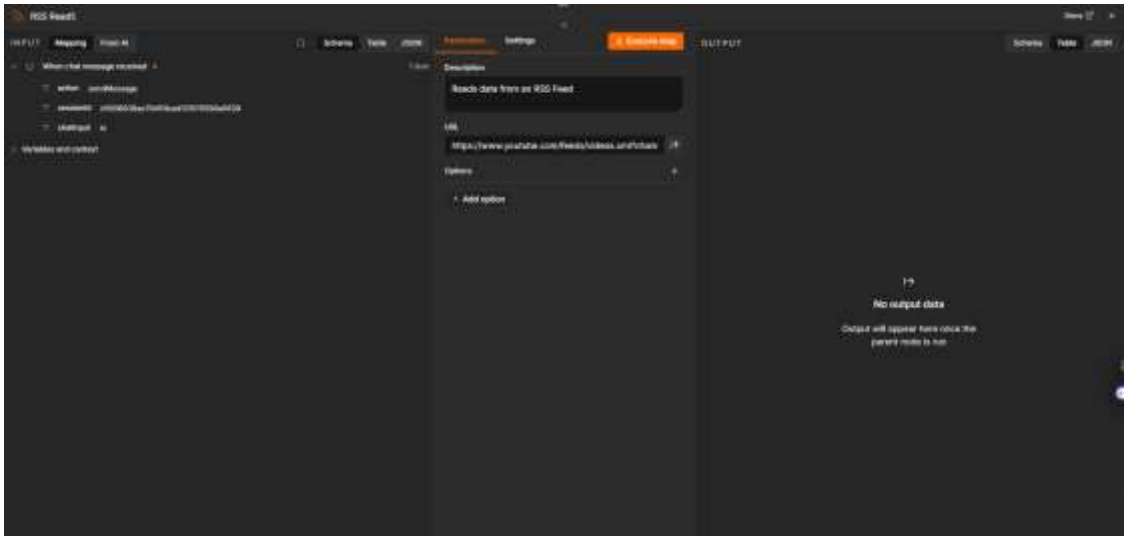


Figure 7. RSS Read5 — an RSS Read tool node configured against a YouTube feed URL.

4. API Samples — OpenRouter

OpenRouter exposes an OpenAI-compatible Chat Completions endpoint. Based on the configured node (Figure 5), this is the request n8n sends on the agent’s behalf, including prior turns pulled from Postgres memory:

```
POST https://openrouter.ai/api/v1/chat/completions
Authorization: Bearer <OPENROUTER_API_KEY>
Content-Type: application/json

{
  "model": "openrouter/free",
  "messages": [
    { "role": "user", "content": "hi there" },
    { "role": "assistant", "content": "Hello! How can I help you today? Would you like me to read
from an RSS feed, or is there something else I can assist you with?" },
    { "role": "user", "content": "hi" }
  ]
}
```

And the corresponding response, taken directly from the node's Output panel in Figure 5:

```
{
  "generations": [[
    {
      "text": "Hi again! \ud83d\udc4b How can I help you today? Feel free to ask me to read an RSS
feed, or let me know what you'd like assistance with.",
      "generationInfo": { "finish_reason": "stop" }
    }
  ]],
  "tokenUsage": { "completionTokens": 64, "promptTokens": 593, "totalTokens": 657 }
}
```

The 657 total tokens match exactly what n8n's execution log reported for this run, confirming this is the real call behind it rather than an illustrative guess.

5. Integration Points

Integration Point	Mechanism	Role in the Workflow
Chat input	n8n Chat Trigger (hosted, public webhook)	Starts the workflow with the user's message, session ID, and action as input (Figure 3).
Reasoning	OpenRouter Chat Model API (POST /chat/completions, model openrouter/free)	Supplies the language model the AI Agent reasons with; every reply is one API round trip (Figure 5).
Persistence	Postgres Chat Memory node → n8n_chat_histories table	Writes and reads conversation history by session ID, giving the agent multi-turn context (Figure 6).
External data retrieval	RSS Read tool nodes (HTTP GET per feed, e.g. a YouTube feed)	Called by the agent on demand to pull live feed content into the conversation (Figure 7).

6. Explanation

Logic: the AI Agent node is the coordination point — it does not itself store data, call the LLM, or fetch feeds; it delegates each of those to the node wired into the matching slot (Memory, Chat Model, Tool) and merges the results into one reply. That separation is what let me swap in Postgres for memory and OpenRouter for the model without touching the agent's own configuration.

Data structure: the schema in Section 2 is intentionally minimal — session_id plus a JSONB message column — because chat history is naturally a sequence of variably-shaped messages (text now, potentially tool calls or attachments later), and JSONB accommodates that without a schema migration every time the message shape changes. The Context Window Length setting (5) is a separate, tunable control over how much of that stored history is replayed to the model each turn.

API understanding: Section 4 shows I can identify the exact request n8n is issuing on my behalf, not just that a node exists — including matching the response's token usage back to what the node's own output panel reported.

Integration: Figures 4 and 6 are evidence, not description — the AI Agent's run log and the Postgres node's stored history both show the pieces working together on a real turn, with the RSS tools available but only called when the agent decides a request needs one.